REPUBLIQUE DU TCHAD
Unité – Travail – Progrès
****
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR
****
UNIVERSITE DE MOUNDOU
****
FACULTE DES SCIENCES EXACTES ET APPLIQEES
****
DEPARTEMENT DES SCIENCES TECHNOLOGIES
BP 206 Moundou, Tchad
Tel: (00235) 65291329
Email: udm@univ-mdou.org
****

جمهورية تشاد
****
وحدة – عمل – تقد
****
وزارة التعليم العالي
****
جامعة مو ندو
****
كلية العلوم الدقيقة والتطبيقية
****
قسم علوم التكنولوجيا

RAPPORT DE TRAVAUX PRATIQUE N° 02

THEME :

Développement web, Gestion d'une ligne de commande

Application web : Laravel, Bootstrap et MySQL (XAMPP)

Conversion mobile réalisée par :

- ✓ **ALI Adoum Abakar**
- ✓ **CHAHAD Banane Moustapha**

Encadrant : **Prof. Arouna NDAM NJOYA**

Date : **Octobre 2026**

Sommaire

Résumé....2
1. Introduction....3
1.1 Contexte....3
1.2 Travail demandé....3
1.3 Résultat obtenu....3
2. Environnement et outils....4
3. Analyse et conception....4
3.1 Besoins fonctionnels....4
3.2 Cas d'utilisation....5
3.3 Modèle de données....6
3.4 Intégrité référentielle....7
3.5 Règles de gestion....8
4. Architecture de l'application....9
4.1 Le modèle MVC de Laravel....9
4.2 Organisation du projet....10
4.3 Plan de navigation....10
5. Réalisation....11
5.1 Mise en place du projet....11
5.2 Base de données et migrations....11
5.3 Modèles Eloquent et relations....12
5.4 Contrôleurs, validation et règles de gestion....12
5.5 Vues Blade et interface Bootstrap....13
5.6 Saisie du département et de la région....13
6. Fonctionnalités et interfaces....13
6.1 Correspondance avec le sujet....13
6.2 Aperçu des pages....14
6.3 Le tableau de bord....15
6.4 Gestion d'une commande....18
7. Gestion du stock et intégrité des données....20
8. Tableau de bord et analyse de performance....21
8.1 Indicateurs choisis....21
8.2 Analyse de performance....22
9. Tests et difficultés rencontrées....22
9.1 Scénarios de test....22
9.2 Difficultés et solutions....23
10. Limites et perspectives....24
10.1 Limites....24
10.2 Perspectives....24
Conclusion....25
Références....26
Annexe A : tableau des routes....27
Annexe B : installation et lancement....28

Résumé

Ce rapport présente la conception et la réalisation d'une application web de **gestion d'une ligne de commande**, développée avec **Laravel**, **Bootstrap** et **MySQL** sous XAMPP, dans le cadre du TP d'examen de développement web du Master 1 DI/IRS de l'Université de Moundou.

L'application permet de gérer les clients, les produits et les commandes. Elle enregistre, modifie et retire les produits d'une commande en maintenant le stock à jour, et offre un tableau de bord statistique accompagné d'une analyse de performance. Le rapport décrit l'analyse, le modèle de données, l'architecture, les choix techniques, les difficultés rencontrées et les perspectives d'évolution.

Mots-clés : Laravel, Eloquent, MVC, Bootstrap, MySQL, transactions, gestion de stock, tableau de bord.

1. Introduction

1.1 Contexte

Ce travail s'inscrit dans le TP d'examen de développement web du Master 1 DI/IRS. Le sujet demande de réaliser une application de **gestion d'une ligne de commande** avec HTML, CSS (Bootstrap) et le framework PHP Laravel, sur une base de données MySQL hébergée par XAMPP.

Il s'agit du premier projet Laravel de l'étudiant. La démarche a donc été progressive : installation de l'environnement, création de la base, des modèles et des vues, puis ajout des fonctionnalités une par une, chacune testée avant de passer à la suivante.

1.2 Travail demandé

Le sujet porte sur six tables (Client, Commande, ligneCommande, Produit, Region, Département) et demande de :

- proposer un **formulaire d'enregistrement des produits d'une commande** donnée ;
- afficher la **liste des produits**, la **liste des commandes** et la **liste des clients** ;
- **modifier un produit** ;
- **supprimer un produit dans une commande** ;
- réaliser un **tableau de bord** de statistiques sur les lignes de commande. Cette tâche est ouverte : les choix doivent être justifiés et une analyse de performance est attendue.

1.3 Résultat obtenu

L'application couvre les cinq points du sujet. Elle ajoute plusieurs fonctions utiles pour l'utiliser sans passer par phpMyAdmin :

- création de clients, de produits et de commandes depuis l'interface (les départements et régions sont saisis dans le formulaire du client) ;
- modification d'une commande sur une seule page (client, date, quantités, produits retirés ou ajoutés) et suppression d'une commande ;
- vérification et mise à jour automatique du stock à chaque opération ;
- menu latéral adaptatif, qui se replie sur petit écran.

2. Environnement et outils

Le tableau suivant récapitule l'environnement de développement utilisé.

Élément	Détail
Système	Windows
Serveur local	XAMPP : Apache 2.4.58, MySQL et phpMyAdmin
Langage	PHP 8.2.12
Framework	Laravel 12.69.3, avec le gestionnaire de dépendances Composer
Interface	HTML5, CSS, Bootstrap 5.3.3 et Bootstrap Icons 1.11.3 (chargés par CDN)
Graphiques	Chart.js 4.4.3 (chargé par CDN)
Éditeur	Visual Studio Code
Base de données	`gestion_commandes` (jeu de caractères utf8mb4)
Compte MySQL	`MasTMound2`, limité à la base du TP (principe du moindre privilège)
Adresse locale	`http://localhost/DevWeb/TP1_exam/tp_exam_commande/public/`

La configuration de la connexion à la base se fait dans le fichier `.env` à la racine du projet (le mot de passe est masqué ici) :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=gestion_commandes
DB_USERNAME=MasTMound2
DB_PASSWORD="********"
```

3. Analyse et conception

3.1 Besoins fonctionnels

Réf.	Fonctionnalité	Description
F1	Enregistrer les produits d'une commande	Choisir un produit et une quantité, avec contrôle du stock
F2	Consulter les listes	Produits, commandes et clients, paginés par 10
F3	Modifier un produit	Désignation, stock et prix unitaire
F4	Supprimer un produit dans une commande	Retirer la ligne et remettre la quantité en stock
F5	Tableau de bord	Indicateurs et graphiques sur les lignes de commande
F6	Gérer les clients	Ajouter et modifier un client, avec son département et sa région
F7	Gérer les commandes	Créer, modifier (client, date, quantités, produits) et supprimer une commande

3.2 Cas d'utilisation

L'application est utilisée par un seul acteur, le **gestionnaire** : le sujet ne demande pas d'authentification. La figure 1 résume les cas d'utilisation.

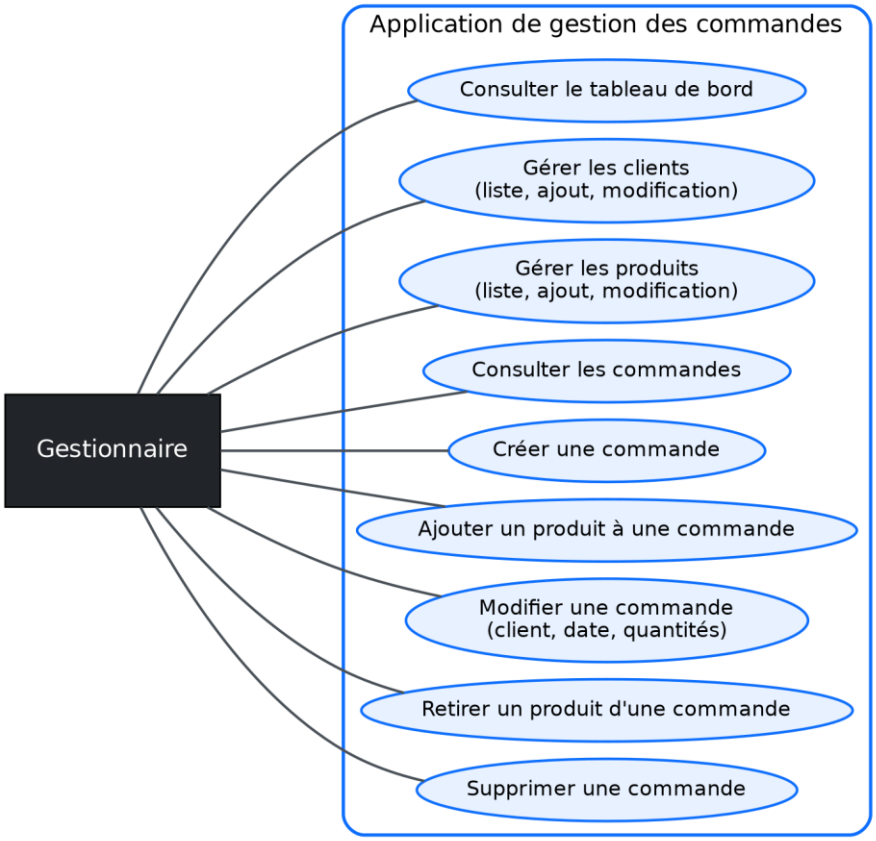


Figure 1 : diagramme des cas d'utilisation

3.3 Modèle de données

Le modèle reprend fidèlement les six tables du sujet (figure 2). Un département contient plusieurs régions, une région regroupe plusieurs clients, et un client passe plusieurs commandes. Une commande contient plusieurs produits et un produit peut figurer dans plusieurs commandes : cette association plusieurs-à-plusieurs est portée par la table **ligneCommande**, qui ajoute la quantité commandée (`qteCmd`) et dont la clé primaire est composée de `idCmd` et `idProd`.

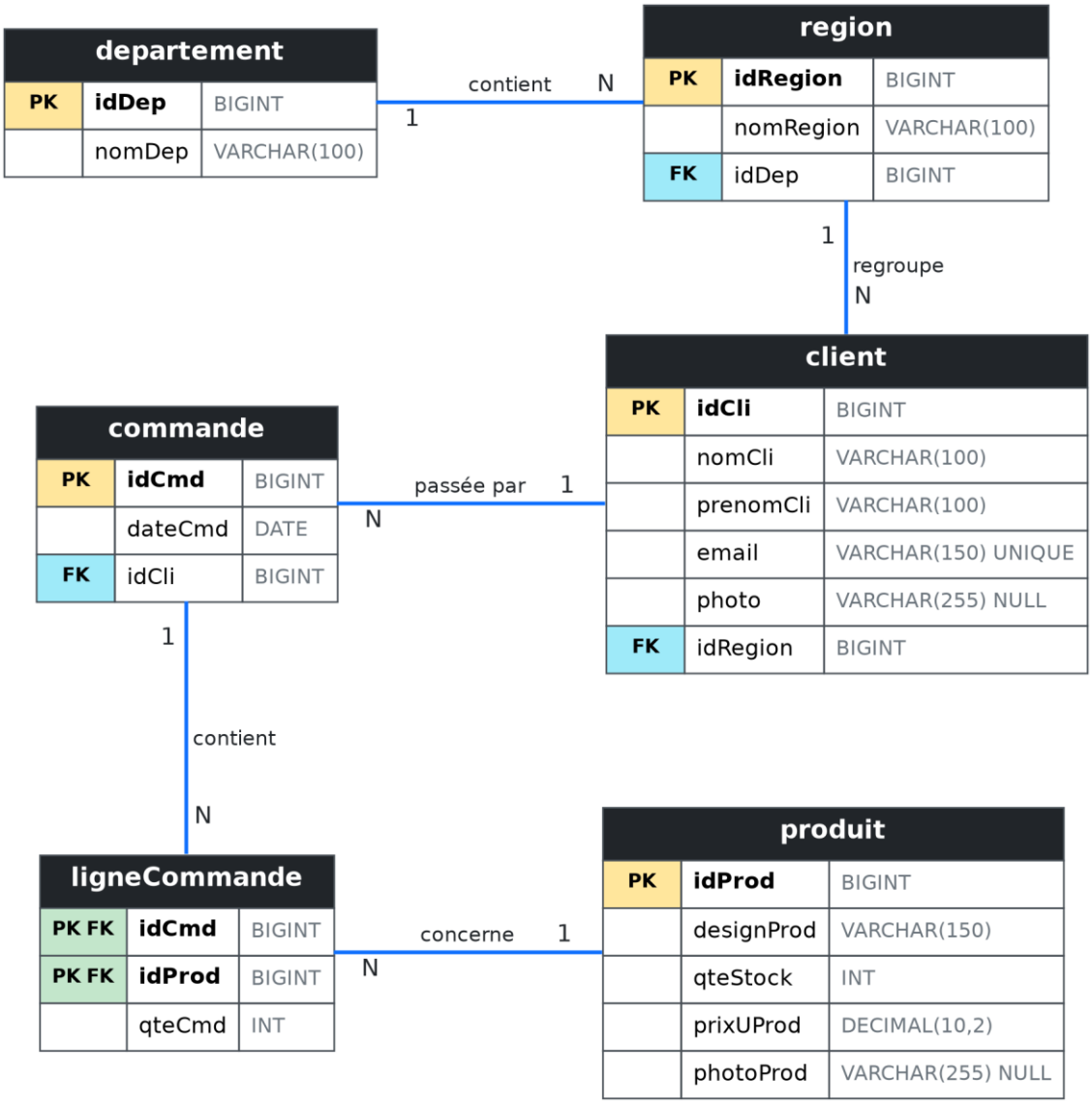


Figure 2 : modèle de données (PK : clé primaire, FK : clé étrangère, 1 et N : cardinalités)

Dictionnaire de données

Table	Colonne	Type	Contrainte
departement	idDep	BIGINT UNSIGNED	Clé primaire, auto-incrémentée
	nomDep	VARCHAR(100)	Obligatoire
region	idRegion	BIGINT UNSIGNED	Clé primaire
	nomRegion	VARCHAR(100)	Obligatoire
	idDep	BIGINT UNSIGNED	Clé étrangère vers departement (RESTRICT)
client	idCli	BIGINT UNSIGNED	Clé primaire
	nomCli, prenomCli	VARCHAR(100)	Obligatoires
	email	VARCHAR(150)	Obligatoire, unique
	photo	VARCHAR(255)	Facultative
	idRegion	BIGINT UNSIGNED	Clé étrangère vers region (RESTRICT)
produit	idProd	BIGINT UNSIGNED	Clé primaire
	designProd	VARCHAR(150)	Obligatoire
	qteStock	INT UNSIGNED	Défaut 0, jamais négatif
	prixUProd	DECIMAL(10,2)	Obligatoire
	photoProd	VARCHAR(255)	Facultative
commande	idCmd	BIGINT UNSIGNED	Clé primaire
	dateCmd	DATE	Obligatoire
	idCli	BIGINT UNSIGNED	Clé étrangère vers client (RESTRICT)
ligneCommande	idCmd	BIGINT UNSIGNED	Clé primaire composée, clé étrangère vers commande (CASCADE)
	idProd	BIGINT UNSIGNED	Clé primaire composée, clé étrangère vers produit (RESTRICT)
	qteCmd	INT UNSIGNED	Quantité commandée

3.4 Intégrité référentielle

Le choix de l'action associée à chaque clé étrangère protège la cohérence des données :

Relation	Suppression	Justification
commande vers ligneCommande	CASCADE	Une ligne n'a aucun sens sans sa commande : elle disparaît avec elle.
produit vers ligneCommande	RESTRICT	Un produit déjà commandé fait partie de l'historique : MySQL refuse de le supprimer.
client vers commande	RESTRICT	L'historique des commandes d'un client est conservé.

Relation	Suppression	Justification
region vers client, departement vers region	RESTRICT	On ne supprime pas une région ou un département encore utilisés.

3.5 Règles de gestion

Réf.	Règle
R1	Un département contient plusieurs régions, une région regroupe plusieurs clients, et chaque client appartient à une seule région.
R2	L'adresse email d'un client est unique.
R3	Le département et la région sont obligatoires à la création et à la modification d'un client. S'ils n'existent pas encore, ils sont créés automatiquement. La recherche ignore la casse : « Savane » et « savane » désignent le même département.
R4	Une commande appartient à un client et porte une date.
R5	Un produit ne peut figurer qu'une seule fois dans une même commande.
R6	La quantité commandée est un entier supérieur ou égal à 1 et ne peut pas dépasser le stock disponible.
R7	Le stock diminue à l'ajout d'un produit. Il augmente au retrait d'un produit ou à la suppression d'une commande. Il est ajusté de la différence lors d'une modification de quantité.
R8	Toute opération qui modifie plusieurs tables s'exécute dans une transaction : elle réussit entièrement ou elle est annulée.

4. Architecture de l'application

4.1 Le modèle MVC de Laravel

Laravel applique le patron **MVC** (Modèle, Vue, Contrôleur) : chaque couche a un rôle précis, ce qui rend le code plus lisible et plus facile à faire évoluer. La figure 3 montre le trajet d'une requête, de la saisie d'une adresse dans le navigateur jusqu'à l'affichage de la page.

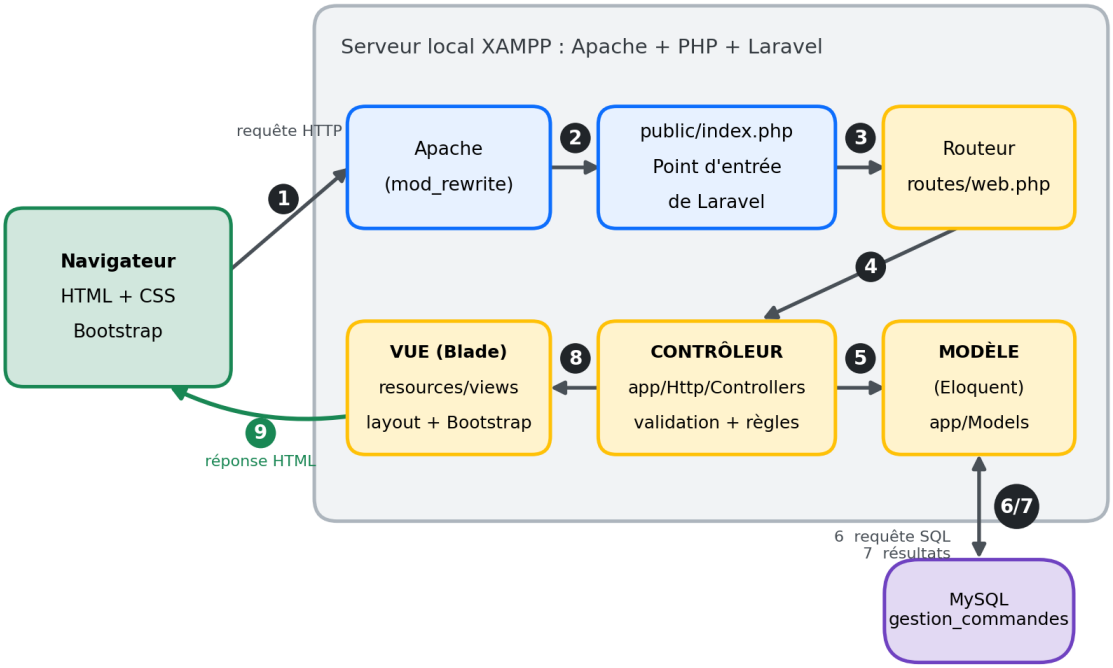


Figure 3 : cycle de traitement d'une requête dans l'application

- **1.** Le navigateur envoie une requête HTTP (par exemple `GET /commandes`).
- **2 et 3.** Apache, grâce à `mod_rewrite` et au fichier `public/.htaccess`, transmet la requête à `public/index.php`, qui démarre Laravel.
- **4.** Le routeur (`routes/web.php`) associe l'adresse à une méthode de contrôleur.
- **5.** Le contrôleur valide les données saisies, applique les règles de gestion et interroge les modèles.
- **6 et 7.** Les modèles Eloquent traduisent ces appels en requêtes SQL ; MySQL renvoie les résultats.
- **8 et 9.** Le contrôleur transmet les données à la vue Blade, qui produit la page HTML (avec Bootstrap) renvoyée au navigateur.

Couche	Rôle	Emplacement
Modèle	Représente une table et ses relations	`app/Models`
Vue	Génère le HTML (Blade et Bootstrap)	`resources/views`
Contrôleur	Reçoit la requête, valide, applique les règles, choisit la vue	`app/Http/Controllers`
Routes	Associe chaque adresse à un contrôleur	`routes/web.php`
Migrations	Décrivent la structure de la base de données	`database/migrations`

4.2 Organisation du projet

```
tp_exam_commande/
├── app/
│   ├── Http/Controllers/
│   │   ├── ClientController.php
│   │   ├── CommandeController.php
│   │   ├── DashboardController.php
│   │   └── ProduitController.php
│   ├── Models/
│   │   ├── Client.php          ├── Commande.php
│   │   ├── Departement.php     ├── Produit.php
│   │   └── Region.php
│   └── Providers/AppServiceProvider.php
├── database/migrations/        (6 tables du sujet)
├── resources/views/
│   ├── layouts/app.blade.php   (menu latéral)
│   ├── dashboard.blade.php
│   ├── clients/     (index, create, edit)
│   ├── produits/    (index, create, edit)
│   └── commandes/   (index, create, show, edit, edit-produit)
├── routes/web.php
└── .env                        (configuration MySQL)
```

4.3 Plan de navigation

Le menu latéral donne accès aux quatre sections de l'application. Il reste fixe à gauche sur grand écran et se replie derrière un bouton sur petit écran (composant `collapse` de Bootstrap). La figure 4 présente les pages et leurs adresses.

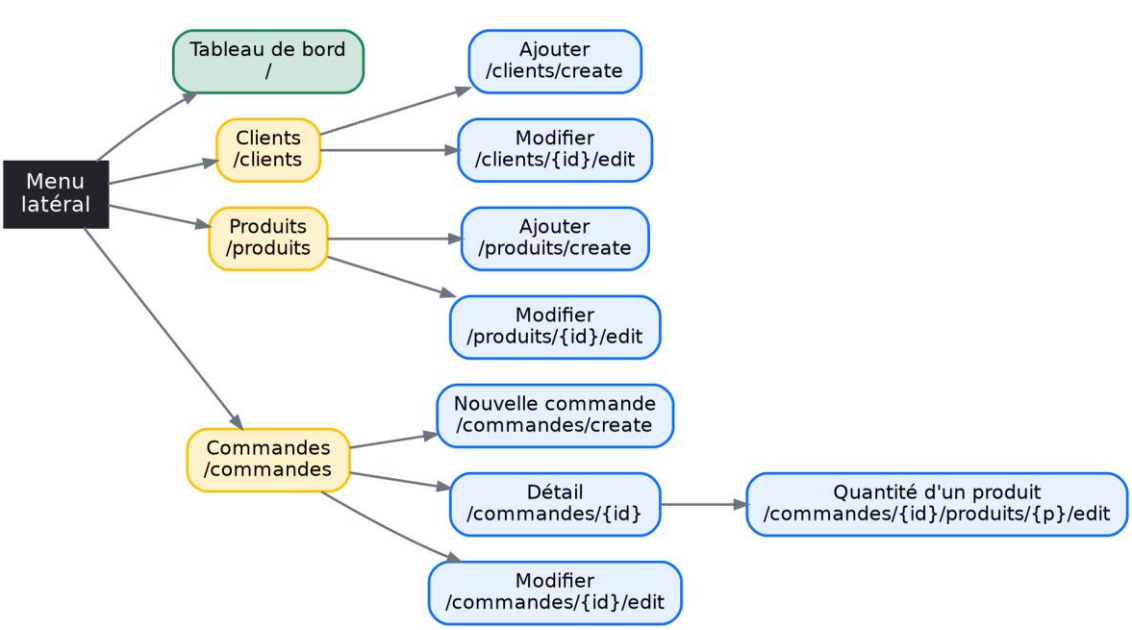


Figure 4 : plan de navigation de l'application

5. Réalisation

5.1 Mise en place du projet

Le projet est créé dans le dossier `htdocs` de XAMPP avec Composer. Les tables sont ensuite créées par des migrations, et les modèles et contrôleurs par la commande `artisan` :

```
cd C:\xampp\htdocs\DevWeb\TP1_exam
composer create-project laravel/laravel tp_exam_commande
cd tp_exam_commande
php artisan make:migration create_departement_table
php artisan make:migration create_region_table
php artisan make:migration create_client_table
php artisan make:migration create_produit_table
php artisan make:migration create_commande_table
php artisan make:migration create_ligneCommande_table
php artisan make:model Departement   # idem Region, Client, Produit, Commande
php artisan make:controller DashboardController   # idem les autres
```

La base et le compte MySQL dédié sont créés dans phpMyAdmin (onglet SQL), puis renseignés dans le fichier `.env` :

```
CREATE DATABASE gestion_commandes CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'MasTMound2'@'localhost' IDENTIFIED BY '********';
GRANT ALL PRIVILEGES ON gestion_commandes.* TO 'MasTMound2'@'localhost';
FLUSH PRIVILEGES;
```

5.2 Base de données et migrations

Une migration est un fichier PHP qui décrit une table. Laravel les exécute dans l'ordre des dates de leurs noms, ce qui impose de créer d'abord les tables référencées : `departement`, `region`, `client`, `produit`, `commande`, puis `ligneCommande`. Les noms de tables et de colonnes sont ceux du sujet. L'extrait suivant montre la table de liaison, avec sa clé primaire composée et ses actions de suppression :

```
Schema::create('ligneCommande', function (Blueprint $table) {
    $table->unsignedBigInteger('idCmd');
    $table->unsignedBigInteger('idProd');
    $table->unsignedInteger('qteCmd');

    // Clé primaire composée : un produit n'apparaît qu'une fois par commande
    $table->primary(['idCmd', 'idProd']);

    // Supprimer une commande supprime ses lignes
    $table->foreign('idCmd')->references('idCmd')->on('commande')
          ->onDelete('cascade');

    // Un produit déjà commandé ne peut pas être supprimé
    $table->foreign('idProd')->references('idProd')->on('produit')
          ->onDelete('restrict');
});
```

Des seeders avaient été envisagés pour remplir la base de données de test. La saisie directe des données depuis l'interface a finalement été retenue, car elle correspond à l'usage réel de l'application.

5.3 Modèles Eloquent et relations

Chaque table a un modèle. Comme les tables du sujet ne suivent pas les conventions de Laravel (noms au singulier, clé `idCli` au lieu de `id`, pas de colonnes `created_at`), chaque modèle déclare `$table`, `$primaryKey` et `$timestamps = false`, ainsi que `$fillable` pour les colonnes modifiables en masse.

Eloquent ne gère pas les clés primaires composées : `ligneCommande` n'a donc pas de modèle. Elle est traitée comme table de liaison par une relation `belongsToMany`, et la colonne supplémentaire `qteCmd` est lue grâce à `withPivot('qteCmd')`. Les lignes sont ajoutées, modifiées et supprimées avec `attach()`, `updateExistingPivot()` et `detach()`.

```
class Commande extends Model
{
    protected $table = 'commande';
    protected $primaryKey = 'idCmd';
    public $timestamps = false;
    protected $fillable = ['dateCmd', 'idCli'];

    public function client(): BelongsTo
    {
        return $this->belongsTo(Client::class, 'idCli', 'idCli');
    }

    // ligneCommande est une table de liaison : relation plusieurs-à-plusieurs
    public function produits(): BelongsToMany
    {
        return $this->belongsToMany(Produit::class, 'ligneCommande', 'idCmd', 'idProd')
                    ->withPivot('qteCmd');
    }
}
```

5.4 Contrôleurs, validation et règles de gestion

Les contrôleurs reçoivent les requêtes, valident les champs avec `$request->validate()` (messages d'erreur en français), appliquent les règles de gestion puis redirigent avec un message de succès. Les formulaires sont protégés contre les attaques CSRF par `@csrf`, et les méthodes PUT et DELETE sont simulées avec `@method`.

L'ajout d'un produit à une commande est l'opération la plus sensible. Elle s'exécute dans une **transaction** : le produit est verrouillé (`lockForUpdate`), on vérifie qu'il n'est pas déjà présent et que le stock suffit, puis on insère la ligne et on diminue le stock. Si une étape échoue, rien n'est enregistré.

```
$erreur = DB::transaction(function () use ($donnees, $commande) {
    // Le verrou évite que deux requêtes consomment le même stock
    $produit = Produit::lockForUpdate()->find($donnees['idProd']);

    if ($commande->produits()->wherePivot('idProd', $produit->idProd)->exists()) {
        return 'Ce produit figure déjà dans la commande.';
    }
    if ($donnees['qteCmd'] > $produit->qteStock) {
        return "Stock insuffisant : il ne reste que {$produit->qteStock} unité(s).";
    }

    $commande->produits()->attach($produit->idProd, ['qteCmd' => $donnees['qteCmd']]);
    $produit->decrement('qteStock', $donnees['qteCmd']);

    return null;   // pas d'erreur
});
```

5.5 Vues Blade et interface Bootstrap

Les vues utilisent le moteur de gabarits **Blade**. Le fichier `layouts/app.blade.php` définit le squelette commun : chargement de Bootstrap, menu latéral, messages de succès et d'erreur. Chaque page l'étend avec `@extends` et injecte son contenu dans `@yield('content')`.

L'interface repose sur les composants Bootstrap suivants : grille responsive (`row` et `col`), cartes, tableaux rayés, badges (état du stock, département, région), formulaires, alertes et boutons colorés (vert pour ajouter, jaune pour modifier, rouge pour supprimer, bleu pour l'action principale). La fonction `old()` conserve la saisie de l'utilisateur lorsqu'un formulaire est refusé, et la pagination est adaptée à Bootstrap par `Paginator::useBootstrapFive()`.

5.6 Saisie du département et de la région

Les départements et les régions ne disposent pas de pages dédiées : ils sont saisis directement dans le formulaire du client, où les deux champs sont obligatoires. Des suggestions (`datalist`) proposent les valeurs déjà enregistrées. À l'enregistrement, `firstOrCreate` retrouve le département ou la région s'ils existent, et les crée sinon, dans une transaction :

```
DB::transaction(function () use ($donnees) {
    // firstOrCreate : retrouve le département s'il existe, sinon le crée
    $departement = Departement::firstOrCreate(['nomDep' => trim($donnees['nomDep'])]);

    $region = Region::firstOrCreate([
        'nomRegion' => trim($donnees['nomRegion']),
        'idDep'     => $departement->idDep,
    ]);

    Client::create([
        'nomCli'    => $donnees['nomCli'],
        'prenomCli' => $donnees['prenomCli'],
        'email'     => $donnees['email'],
        'idRegion'  => $region->idRegion,
    ]);
});
```

6. Fonctionnalités et interfaces

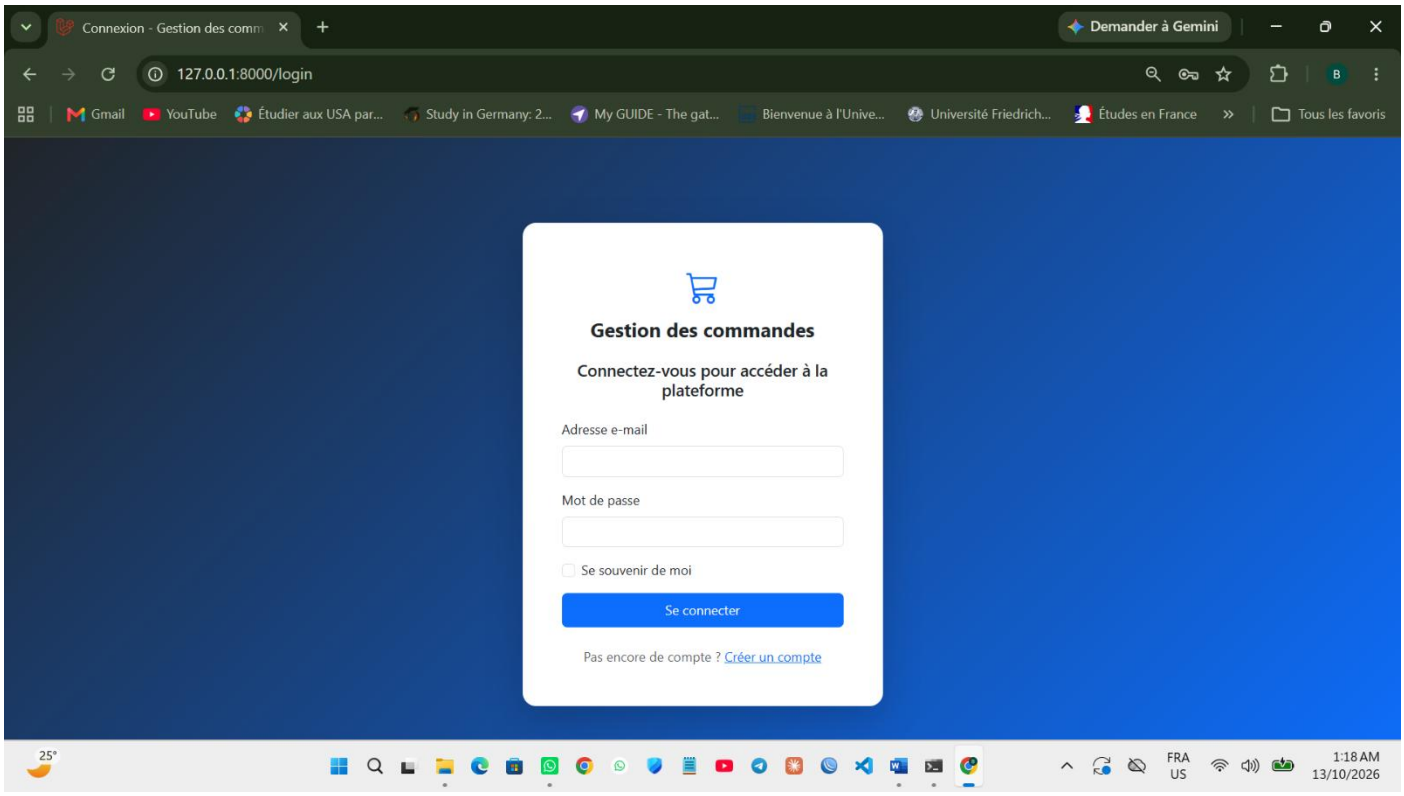
6.1 Correspondance avec le sujet

Exigence du sujet	Réalisation
1. Formulaire d'enregistrement des produits d'une commande	Page `/commandes/{id}` : formulaire produit et quantité, traité par `CommandeController@ajouterProduit`. Le stock est contrôlé et mis à jour.
2. Listes des produits, commandes et clients	Pages `/produits`, `/commandes` et `/clients`, paginées par 10. La liste des commandes affiche aussi le département, la région et le total de chaque commande.
3. Modifier un produit	Page `/produits/{id}/edit` : désignation, stock et prix, validés par `ProduitController@update`.
4. Supprimer un produit dans une commande	Bouton Retirer (puis case à cocher dans la page de modification) : `detach()` et remise de la quantité en stock.
5. Tableau de bord	Page d'accueil `/` : `DashboardController@index` (section 8).

6.2 Aperçu des pages

Page	Adresse	Contenu	Actions
Tableau de bord	`/`	Indicateurs, graphiques, classements	Lien vers les produits à réapprovisionner
Clients	`/clients`	Nom, prénom, email, région, département	Ajouter, Modifier
Produits	`/produits`	Désignation, prix, stock (badge rouge sous 20)	Ajouter, Modifier
Commandes	`/commandes`	Date, client, département, région, nombre de produits, total	Nouvelle commande, Voir, Modifier
Détail d'une commande	`/commandes/{id}`	Lignes de la commande, total, formulaire d'ajout	Ajouter un produit, Modifier la quantité, Supprimer la commande
Modification d'une commande	`/commandes/{id}/edit`	Formulaire prérempli : client, date, quantités, produits à retirer, produit à ajouter	Mettre à jour, Annuler, Supprimer la commande

6.3 Page de connexion



6.4 Le tableau de bord

La page d'accueil regroupe quatre cartes d'indicateurs (nombre de commandes, chiffre d'affaires, panier moyen, produits à stock faible), trois graphiques Chart.js (chiffre d'affaires par mois, par département et top 5 des produits), le classement des meilleurs clients et la liste des produits à réapprovisionner.

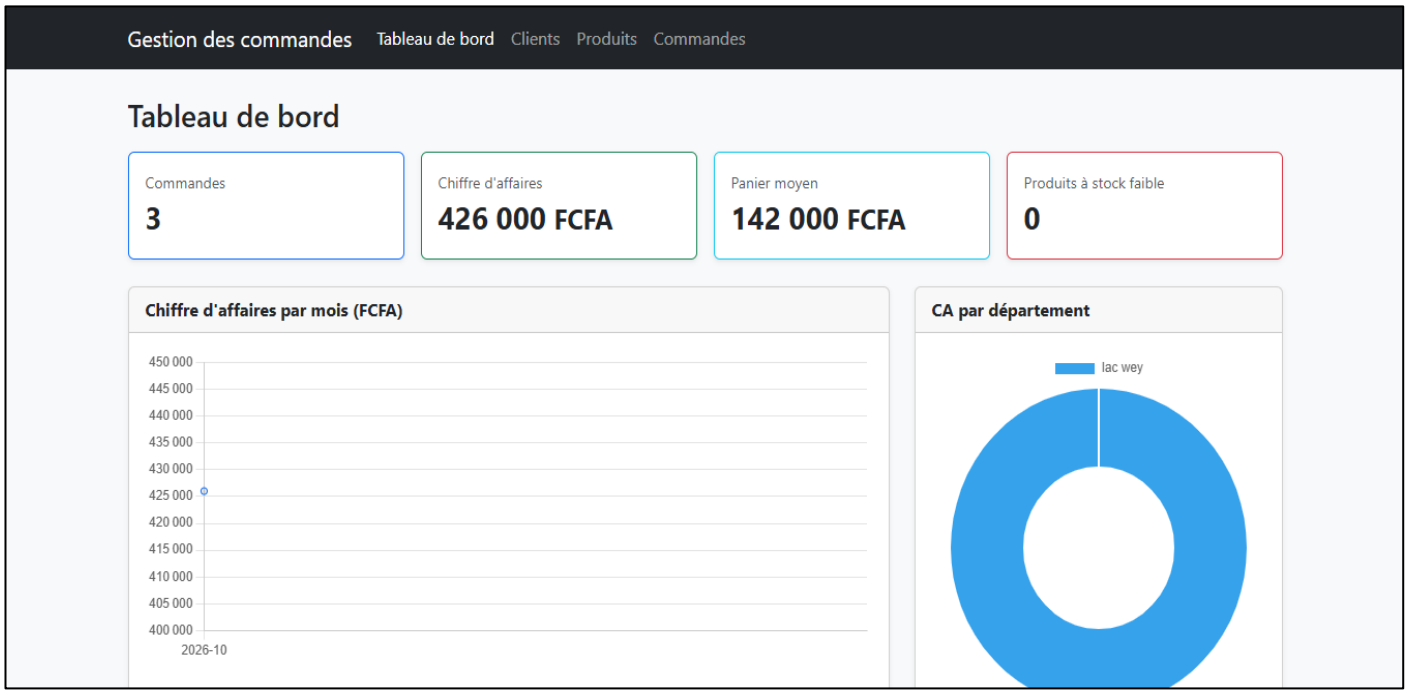
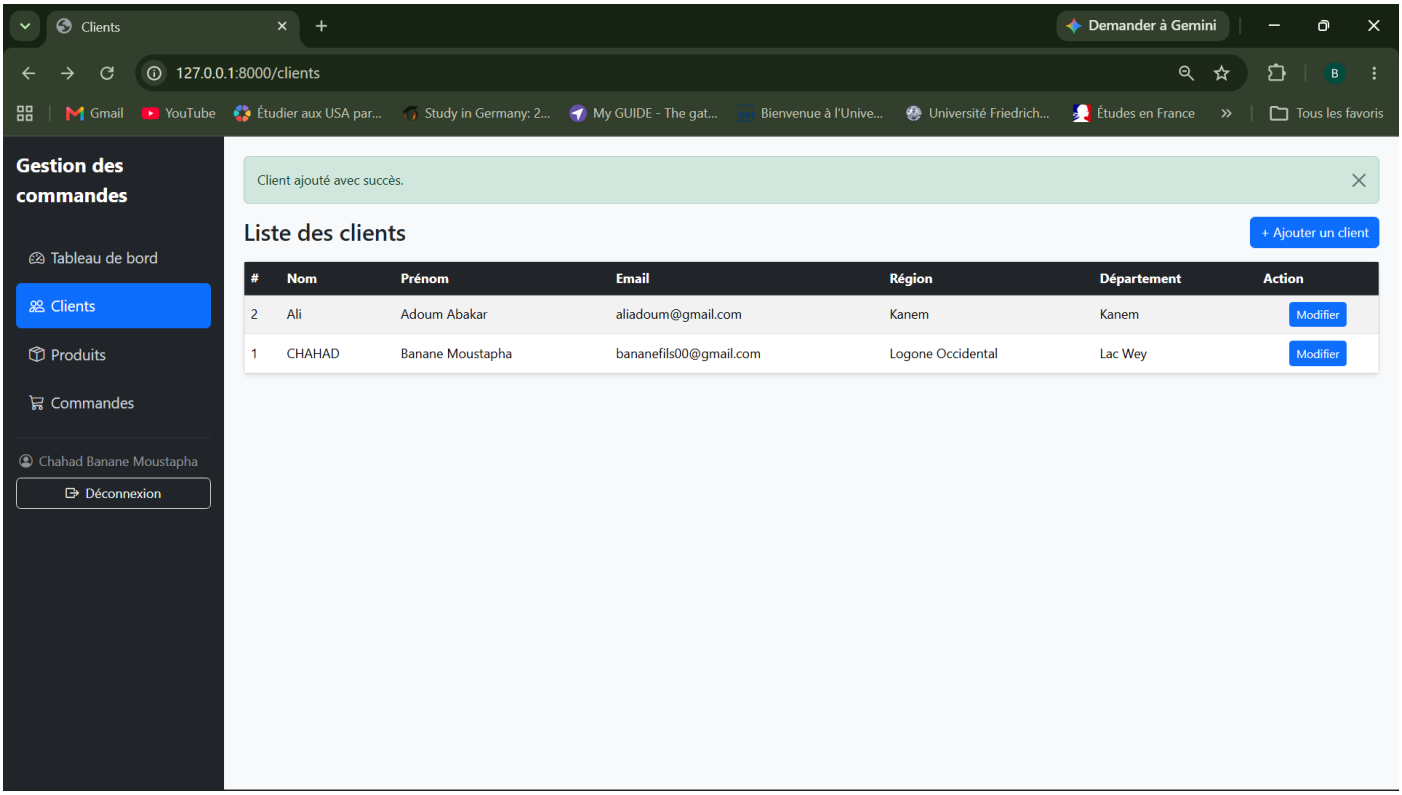
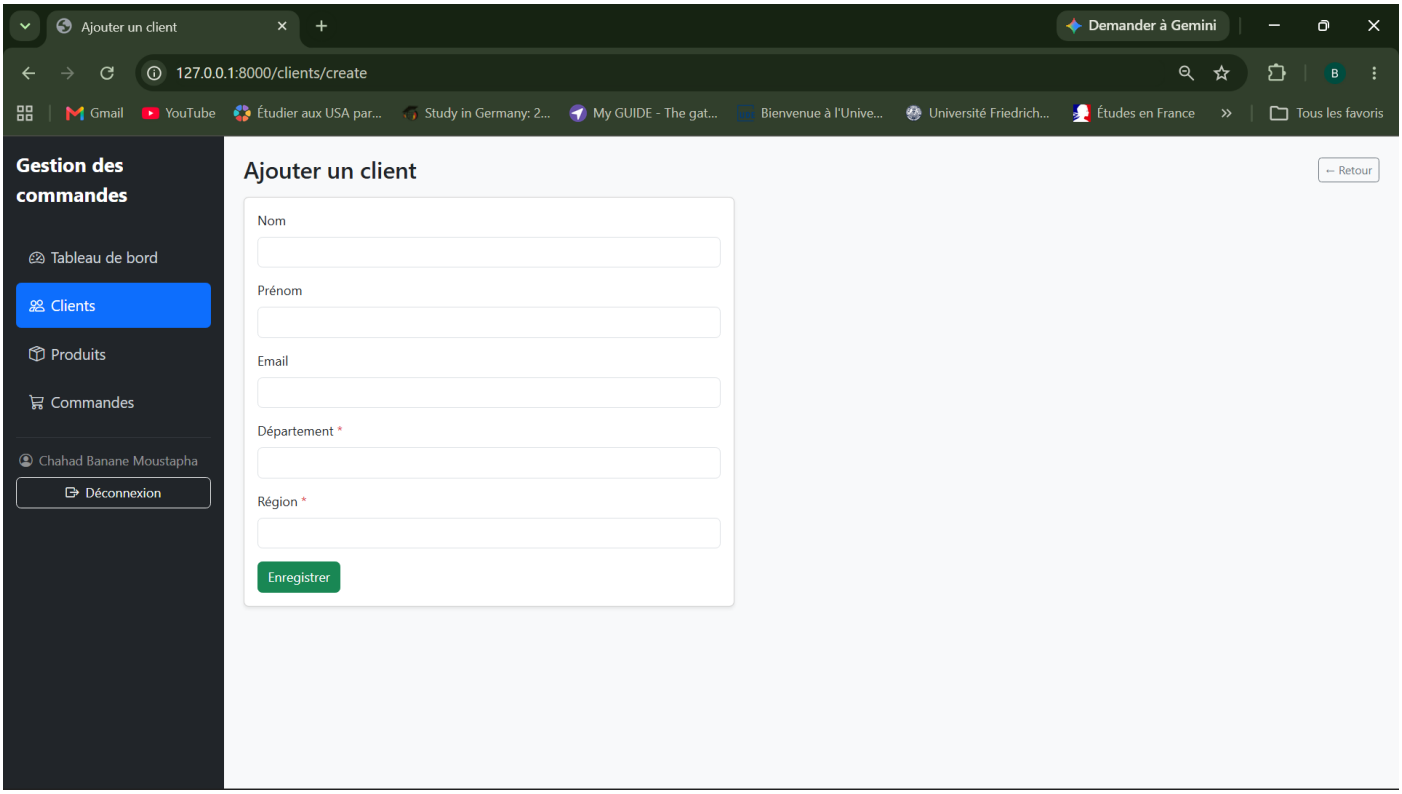


Figure 5 : tableau de bord (jeu de test de 3 commandes). Capture de la première version, à barre de navigation horizontale.

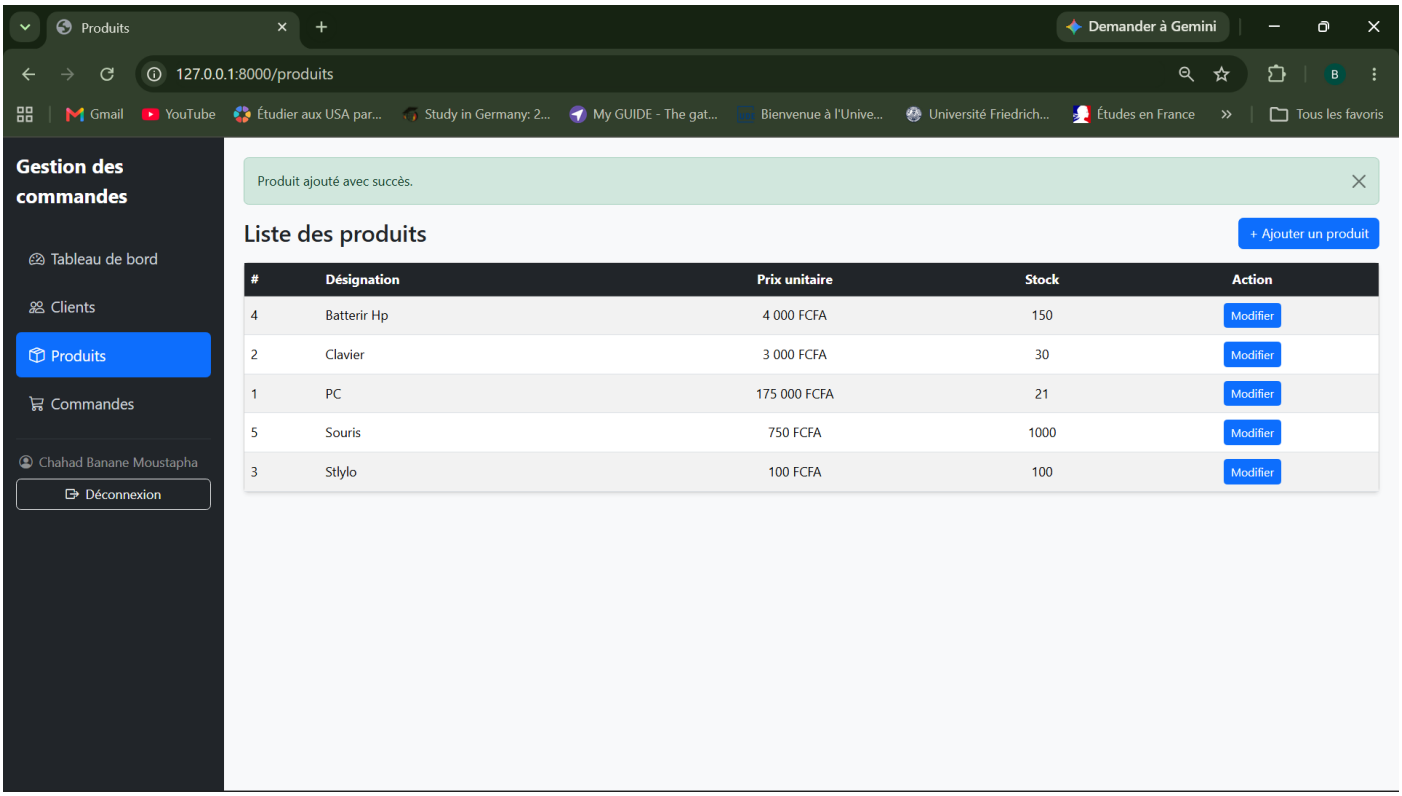
6.5 Liste des clients



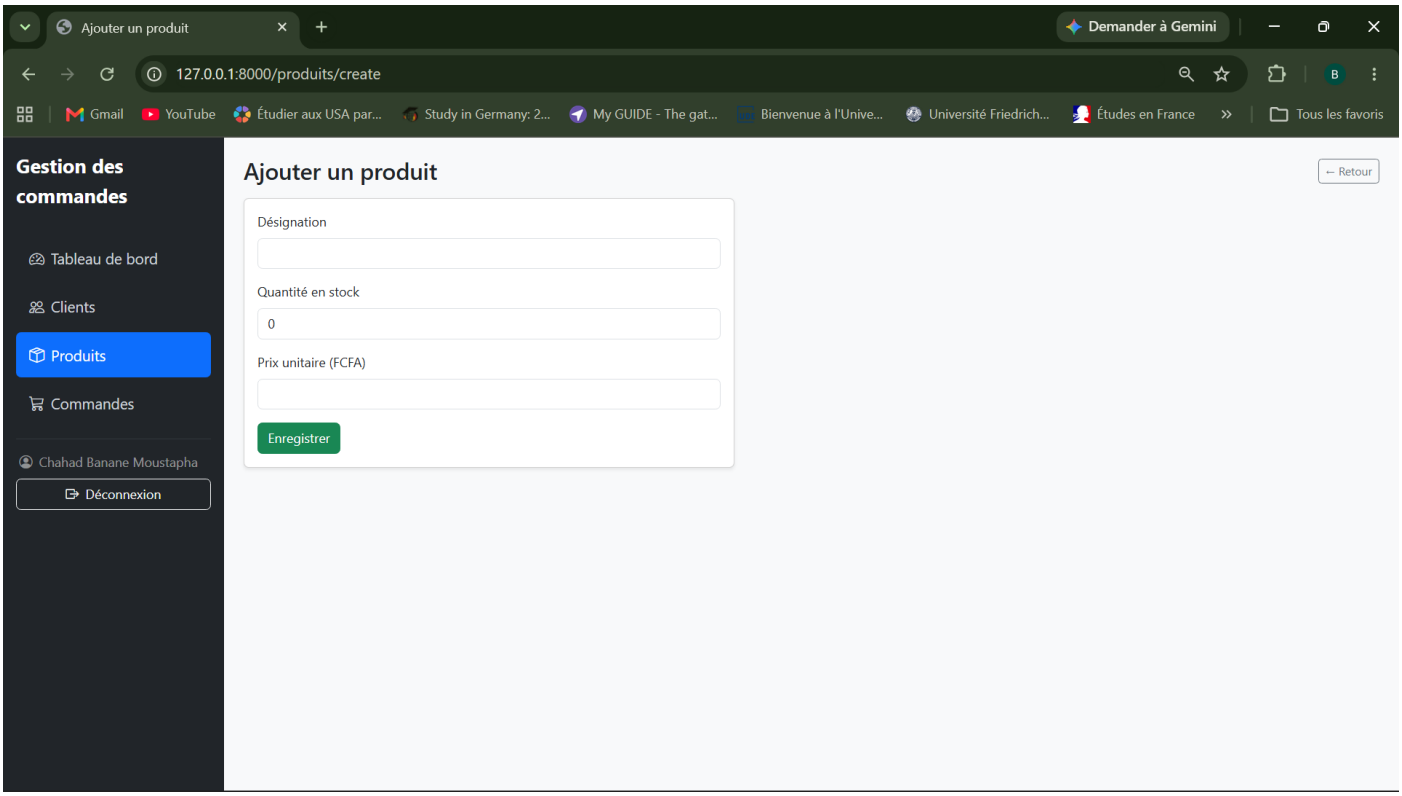
6.6 Formulaire d'ajout d'un nouveau client



6.7 Liste des produits



6.8 Formulaire d'ajout d'un nouveau produit



6.9 Gestion d'une commande

La page d'une commande présente ses lignes (produit, prix unitaire, quantité, sous-total), le total, et un formulaire pour ajouter un produit. La liste déroulante ne propose que les produits en stock qui ne figurent pas déjà dans la commande. La page de modification (`/commandes/{id}/edit`) fonctionne comme celle des clients : formulaire prérempli et bouton « Mettre à jour », avec toutes les modifications enregistrées en une seule transaction.

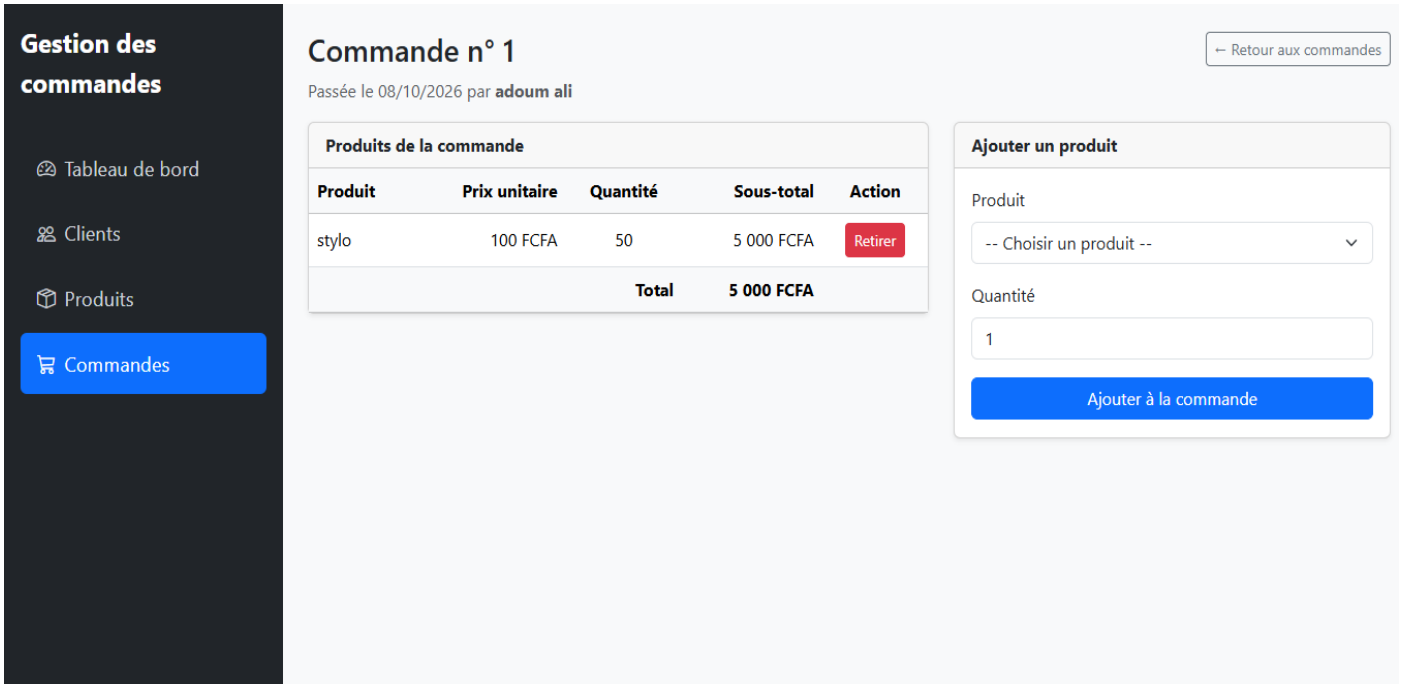
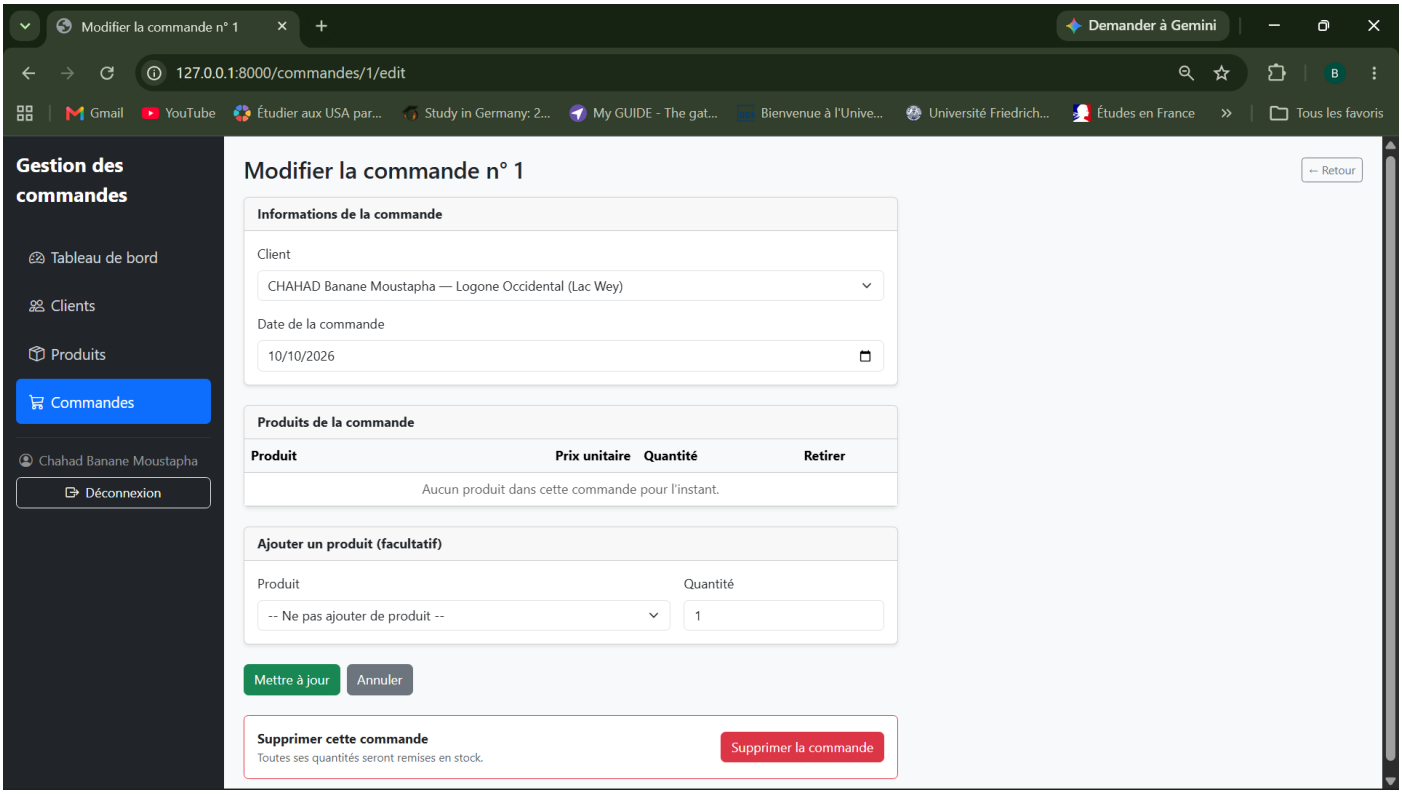
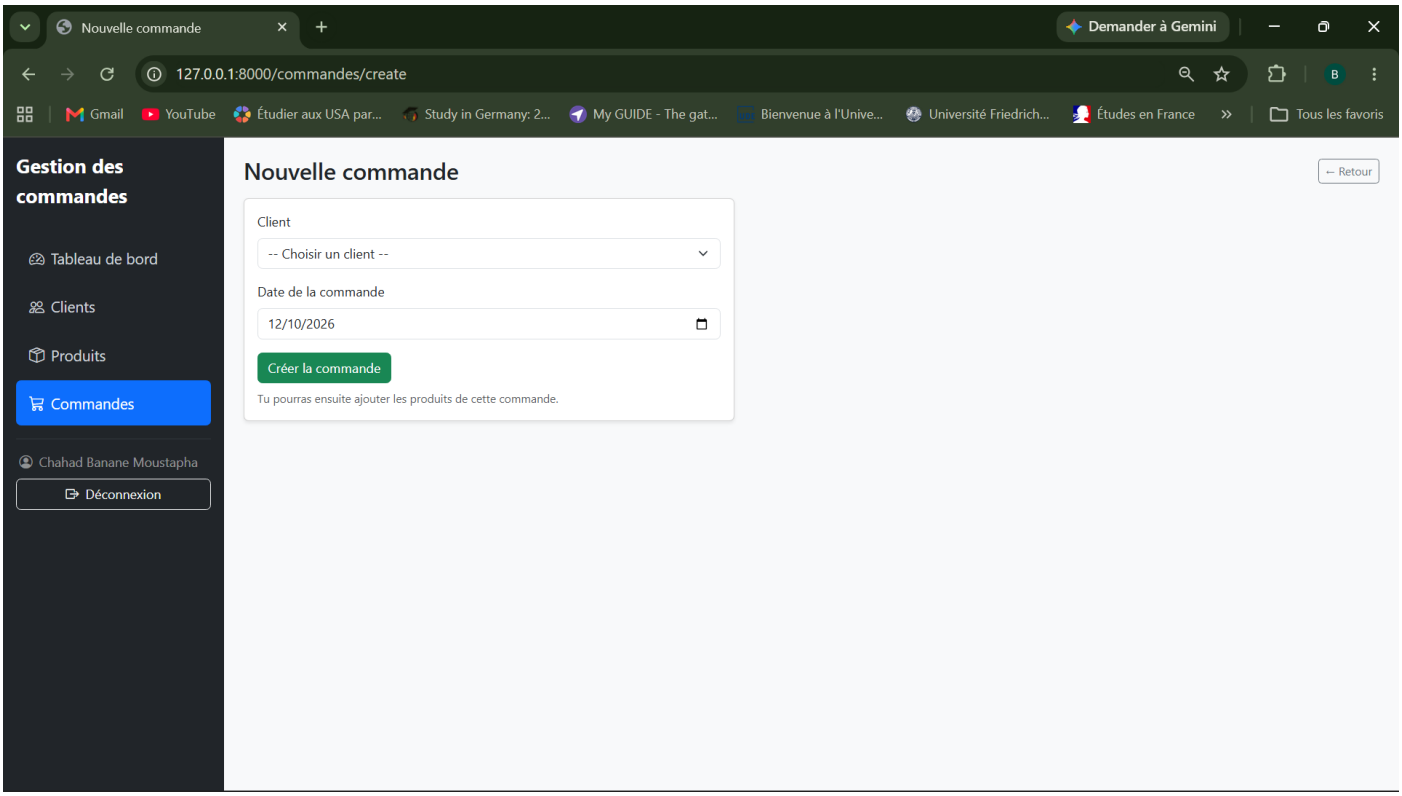


Figure 6 : page d'une commande avec le menu latéral (le bouton « Retirer » a ensuite été remplacé par « Modifier »).

6.9.1 Modification d'une commande existante



6.9.2 Formulaire d’ajout d’un nouveau produit



7. Gestion du stock et intégrité des données

Chaque opération sur une commande met le stock à jour. Les modifications de plusieurs tables sont regroupées dans une transaction : si une étape échoue, la base revient à son état initial. Le verrou posé sur le produit (`lockForUpdate`) empêche deux utilisateurs simultanés de consommer le même stock. La figure 7 détaille l'ajout d'un produit.

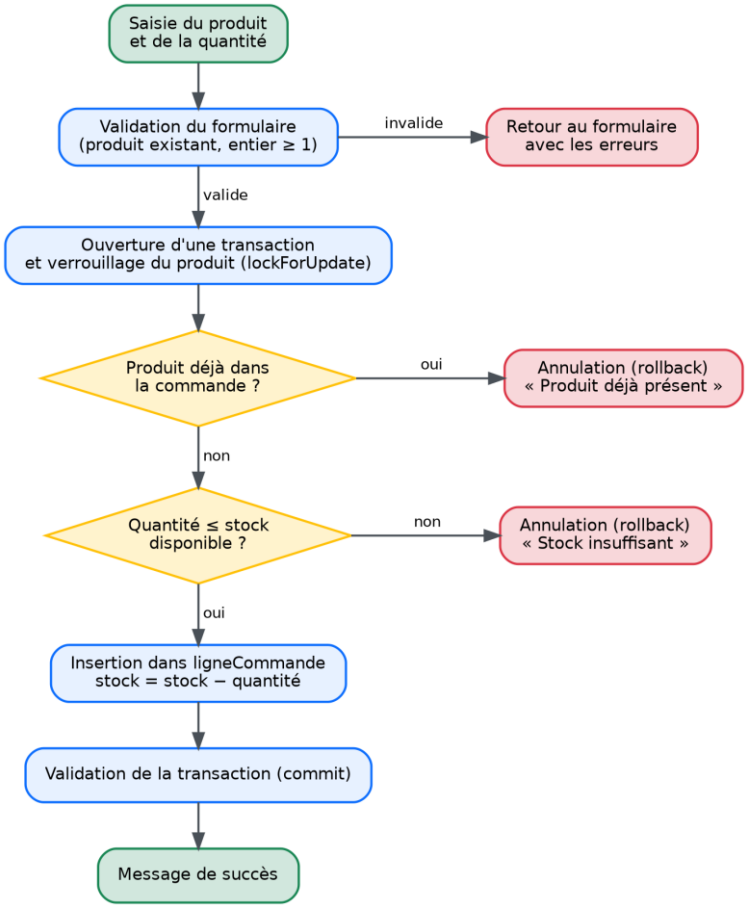


Figure 7 : organigramme de l'ajout d'un produit à une commande

Opération	Effet sur le stock
Ajouter un produit à une commande	stock diminué de la quantité ; refusé si la quantité dépasse le stock
Modifier la quantité d'une ligne	stock diminué de (nouvelle quantité − ancienne quantité) ; refusé si la différence dépasse le stock
Retirer un produit d'une commande	stock augmenté de la quantité de la ligne
Supprimer une commande	stock augmenté des quantités de toutes ses lignes
Mettre à jour une commande (formulaire complet)	tous les ajustements sont appliqués dans une seule transaction (tout ou rien)

8. Tableau de bord et analyse de performance

8.1 Indicateurs choisis

Le sujet laisse libre le contenu du tableau de bord. Chaque indicateur retenu répond à une décision que le gestionnaire peut prendre :

Indicateur	Décision permise	Technique
Nombre de commandes et de clients	Mesurer l'activité	`COUNT`
Chiffre d'affaires et panier moyen	Suivre la santé financière	`SUM(qteCmd × prixUProd)` ; CA divisé par le nombre de commandes
CA par mois (courbe)	Repérer la tendance et la saisonnalité	`GROUP BY` mois
Top 5 des produits vendus (barres)	Décider quoi réapprovisionner	`SUM(qteCmd)` par produit
CA par département (anneau)	Voir la répartition géographique	Jointures jusqu'au département
Top 5 des clients	Identifier les clients à fidéliser	`GROUP BY` client
Produits à stock faible (moins de 20)	Éviter les ruptures	`WHERE qteStock < 20`

Le chiffre d'affaires par département suppose de remonter de la ligne de commande jusqu'au département (figure 8). Les calculs sont faits par MySQL avec des requêtes d'agrégation, comme celle du chiffre d'affaires mensuel :

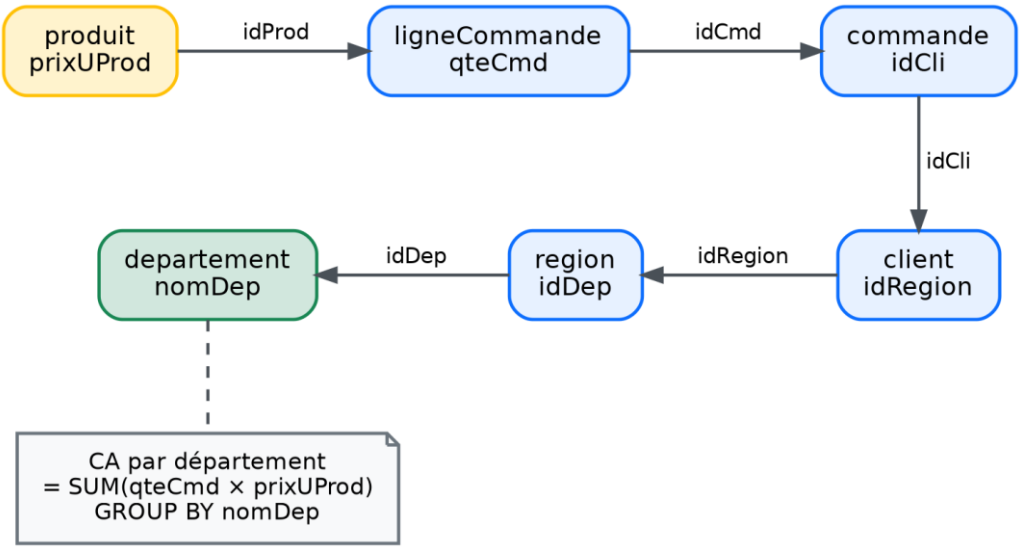


Figure 8 : chemin de jointure du chiffre d'affaires par département

```
$parMois = DB::table('ligneCommande as lc')
    ->join('produit as p', 'p.idProd', '=', 'lc.idProd')
    ->join('commande as c', 'c.idCmd', '=', 'lc.idCmd')
    ->selectRaw("DATE_FORMAT(c.dateCmd, '%Y-%m') as mois,
                 SUM(lc.qteCmd * p.prixUProd) as ca")
    ->groupBy('mois')
    ->orderBy('mois')
    ->get();
```

8.2 Analyse de performance

- **Agrégation en SQL plutôt qu'en PHP.** Les huit requêtes du tableau de bord laissent MySQL calculer les sommes et les regroupements, et ne renvoient que les résultats. Charger toutes les lignes de commande pour les additionner en PHP transférerait un volume qui grandit avec les données, jusqu'à saturer la mémoire.
- **Chargement anticipé contre le problème N+1.** `with()` charge les relations en une seule requête supplémentaire. Pour afficher 10 commandes avec leur client, on passe de 11 requêtes (une pour les commandes, puis une par client) à 2. La différence se mesure avec `DB::enableQueryLog()` dans Tinker.
- **Index.** MySQL indexe automatiquement les clés étrangères, donc les jointures sont déjà rapides. Un index sur `commande.dateCmd` est recommandé pour les regroupements par mois sur de gros volumes.
- **Pagination.** Les listes affichent 10 lignes par page au lieu de tout charger.
- **Mesure intégrée.** Le tableau de bord affiche le nombre de requêtes SQL et son temps de calcul, ce qui permet de comparer des variantes avec des chiffres.
- **Cache (piste non activée).** `Cache::remember` permettrait de ne recalculer les statistiques qu'une fois par minute. Il n'a pas été activé pour que les chiffres restent à jour pendant les tests.

9. Tests et difficultés rencontrées

9.1 Scénarios de test

Scénario	Résultat attendu
Ajouter un produit avec une quantité inférieure ou égale au stock	La ligne est ajoutée, le stock diminue, le total est recalculé.
Ajouter un produit avec une quantité supérieure au stock	Message « Stock insuffisant », rien n'est modifié.
Ajouter un produit déjà présent	Impossible : il n'apparaît plus dans la liste déroulante et le serveur le refuse aussi.
Diminuer la quantité d'une ligne	Le stock augmente de la différence.
Cocher « Retirer » puis « Mettre à jour »	La ligne est supprimée et sa quantité retourne en stock.
Supprimer une commande	La commande et ses lignes disparaissent, le stock est restitué et le tableau de bord est recalculé.
Créer un client avec un email déjà utilisé	Le formulaire est refusé avec un message.
Créer un client sans département ou sans région	Le formulaire est refusé.
Saisir un département avec une casse différente	Le département existant est réutilisé, sans doublon.

9.2 Difficultés et solutions

Problème	Cause	Solution
Liste de fichiers « Index of » à la place de l'application	Le projet était ouvert à la racine du dossier	Ouvrir l'adresse du dossier `public`
Pages vides et indicateurs à 0	La base de données ne contenait aucune donnée	Saisir les données depuis les formulaires de l'application
Erreur « Call to undefined method … create() »	Les routes avaient été ajoutées sans mettre à jour les contrôleurs	Remplacer les contrôleurs en entier et vérifier que chaque route a sa méthode
Clé primaire composée de `ligneCommande`	Eloquent ne la gère pas	Relation `belongsToMany` avec `withPivot`, et `attach`, `detach`
Pagination mal affichée	Laravel utilise Tailwind par défaut	`Paginator::useBootstrapFive()`
Champs département et région vides à la modification d'un client	Lecture d'une propriété inexistante sur le client	Lire les valeurs par les relations : `region` puis `departement`

10. Limites et perspectives

10.1 Limites

- Aucune authentification : le sujet ne la demande pas, mais une application réelle devrait protéger l'accès.
- Les colonnes photo existent et les images s'affichent si elles sont renseignées, mais le téléversement des photos n'est pas implémenté.
- Les clients et les produits ne peuvent pas être supprimés depuis l'interface.
- Il n'y a pas de tests automatisés : les vérifications ont été faites manuellement (section 9.1).
- Les statistiques sont recalculées à chaque affichage.

10.2 Perspectives

- Ajouter une authentification (par exemple Laravel Breeze) et des rôles.
- Gérer le téléversement des photos des clients et des produits (`php artisan storage:link`).
- Mettre en cache les statistiques et ajouter l'index sur `dateCmd`.
- Exporter les listes et les commandes en Excel ou en PDF.
- Proposer une **version mobile** : une application Flutter reprenant les mêmes fonctions et la même interface, sans Laravel, avec une base SQLite locale, est à l'étude.

Conclusion

Ce TP a permis de réaliser une application web complète de gestion d'une ligne de commande, de la base de données au tableau de bord. Les cinq exigences du sujet sont couvertes, et l'application ajoute la gestion des clients, la modification et la suppression d'une commande, ainsi qu'un suivi du stock fiable grâce aux transactions.

Sur le plan pédagogique, ce premier projet Laravel a permis de maîtriser le cycle complet d'une application : migrations, modèles Eloquent et relations, routes, contrôleurs, validation, vues Blade et interface Bootstrap. Il a aussi conduit à des choix de conception justifiés, comme le traitement de la clé primaire composée, l'intégrité référentielle ou l'agrégation des statistiques en SQL.

Références

- Documentation de Laravel : https://laravel.com/docs
- Documentation de Bootstrap : https://getbootstrap.com/docs
- Documentation de Chart.js : https://www.chartjs.org/docs
- XAMPP (Apache Friends) : https://www.apachefriends.org

Annexe A : tableau des routes

Les routes sont déclarées dans `routes/web.php`. Les routes statiques (`/create`) sont déclarées avant les routes à paramètre.

Méthode	Adresse	Nom	Action
GET	`/`	`dashboard`	DashboardController@index
GET	`/produits`	`produits.index`	ProduitController@index
GET	`/produits/create`	`produits.create`	ProduitController@create
POST	`/produits`	`produits.store`	ProduitController@store
GET	`/produits/{produit}/edit`	`produits.edit`	ProduitController@edit
PUT	`/produits/{produit}`	`produits.update`	ProduitController@update
GET	`/clients`	`clients.index`	ClientController@index
GET	`/clients/create`	`clients.create`	ClientController@create
POST	`/clients`	`clients.store`	ClientController@store
GET	`/clients/{client}/edit`	`clients.edit`	ClientController@edit
PUT	`/clients/{client}`	`clients.update`	ClientController@update
GET	`/commandes`	`commandes.index`	CommandeController@index
GET	`/commandes/create`	`commandes.create`	CommandeController@create
POST	`/commandes`	`commandes.store`	CommandeController@store
GET	`/commandes/{commande}`	`commandes.show`	CommandeController@show
GET	`/commandes/{commande}/edit`	`commandes.edit`	CommandeController@edit
PUT	`/commandes/{commande}`	`commandes.update`	CommandeController@update
DELETE	`/commandes/{commande}`	`commandes.destroy`	CommandeController@destroy
POST	`/commandes/{commande}/produits`	`commandes.produits.store`	CommandeController@ajouterProduit
GET	`/commandes/{commande}/produits/{produit}/edit`	`commandes.produits.edit`	CommandeController@editerProduit
PUT	`/commandes/{commande}/produits/{produit}`	`commandes.produits.update`	CommandeController@modifierProduit
DELETE	`/commandes/{commande}/produits/{produit}`	`commandes.produits.destroy`	CommandeController@retirerProduit

Annexe B : installation et lancement

Pour lancer le projet sur un poste équipé de XAMPP : démarrer Apache et MySQL, créer la base et le compte MySQL (section 5.1), configurer le fichier `.env`, puis exécuter dans un terminal :

```
cd C:\xampp\htdocs\DevWeb\TP1_exam\tp_exam_commande
composer install
php artisan config:clear
php artisan migrate

REM puis ouvrir dans le navigateur :
REM http://localhost/DevWeb/TP1_exam/tp_exam_commande/public/
```